Assignment 1 Report

GPU Programming

Problem 4: Cryptography on GPU (Parallel SHA-256 on CUDA)

Professor Binod Kumar
Source code https://github.com/ananpal/gpu_assignment

Indian Institute of Technology Jodhpur

Department of Artificial Intelligence

Submitted by

Group 29

Anand Pal (G25AIT1019)	g25ait1019@iitj.ac.in
Arundhati (G25AIT1033)	g25ait1033@iitj.ac.in
Karan Kapoor (G25AIT1233)	g25ait1233@iitj.ac.in
Mohshinsha Harunsha Shahmadar (G25AIT1093)	g25ait1093@iitj.ac.in
Mudrik Kaushik (G25AIT1096)	g25ait1096@iitj.ac.in

June 30, 2026

1 Abstract

We implement batch SHA-256 by mapping one thread to one message: packed inputs, offset/length index arrays, and disjoint 32-byte digest slots eliminate inter-thread dependencies and race conditions. A reusable host engine allocates memory, performs H2D/D2H copies, launches the kernel, and performs error-safe cleanup; correctness is checked against a CPU reference and `expected_digests.bin` before benchmarking. On one million messages (RTX 4060 Laptop GPU), we observe ~18× and ~63× speedup for end-to-end and kernel-only execution, respectively, compared to a serial CPU baseline. Numbers are from `results/benchmark_run.txt` (`results/scale_benchmark.cu`).

2 Introduction

Problem 4 asks us to speed up a cryptographic primitive on the GPU. We chose batch SHA-256 because hashing N messages is a *map* operation: the same computation applies to every message with no reduction, shared partial state, or read-after-write dependencies between messages. Each thread runs the full hash (padding and compression rounds) in private registers and local memory; parallelism is *across* messages, not within one hash.

This report covers our CUDA parallelization strategy, implementation decisions, and measured results. Scope: single GPU, batch mode, serial CPU baseline via OpenSSL for validation and a scalar reference in the scale harness. Out of scope: multi-GPU, within-message parallelism, and streaming or persistent kernels.

3 Parallelization and CUDA Implementation

3.1 Parallelization Strategy

Alternative considered: parallelize compression rounds or blocks *within* one message—rejected because each round depends on the previous state; threads would need barriers and shared memory with little gain for short messages.

Chosen strategy: one thread per message. Thread i (when $i < N$) reads `offsets[i]` and `lengths[i]`, calls `sha256_hash(messages + offset, len, digests + i*32)`, and writes a 32-byte digest to a slot only it owns. Remainder threads ($i \geq N$) exit immediately. Messages and digests are fully independent. Hence we do not need any synchronization related APIs. Neither we need of `__syncthreads()` nor atomics are needed.

Table 1: Kernel launch parameters (fixed in our implementation).

Parameter	Value
Threads per block (`blockDim.x`)	256
Number of blocks (`gridDim.x`)	$\lceil N/256 \rceil$
Total threads launched	$256 \times \lceil N/256 \rceil \geq N$
Active threads	N

We use 256 threads per block as a default that balances occupancy and register pressure. Please check the (Section 4) for kernel time and throughput.

3.2 Architecture and Data Flow

The project splits into a linkable GPU engine and host tools:

- `sha256_gpu.cu` — `sha256_gpu_hash()`: allocation, copies, launch, cleanup (no `main()`).
- `sha256.cuh` — device functions and `sha256_kernel` (`__global__` entry).
- Host drivers — `cpu_reference`, `hash_dataset`, `validate`, and `benchmark`.

Shared types and layout are defined in `include/sha256_gpu.hpp` and `IO_CONTRACT.md`.

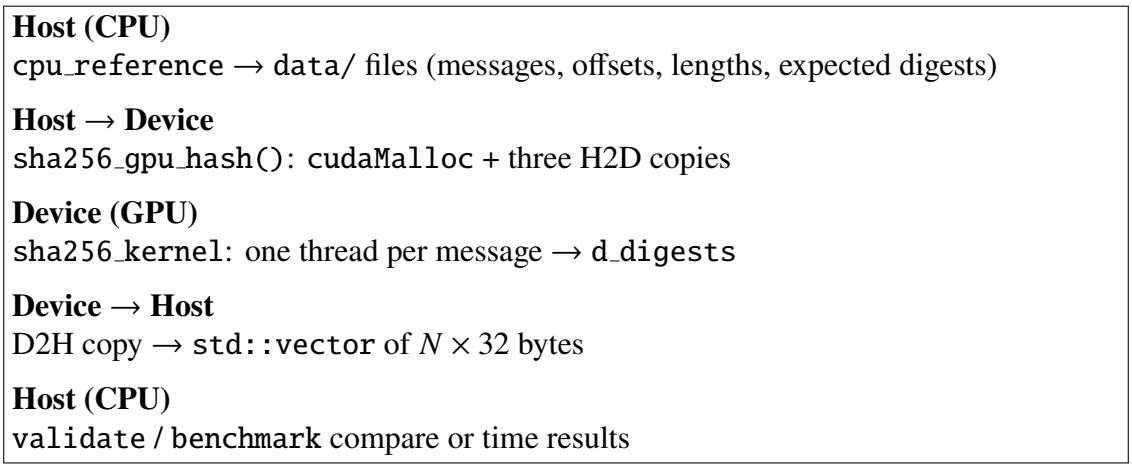

Figure 1: Host–device data flow for batch SHA-256.

All messages live in one packed `messages.bin` buffer; `offsets` and `lengths` index into it. Thread i writes `d_digests[i*32 .. i*32+31]`. After `cudaDeviceSynchronize()`, one D2H `cudaMemcpy` returns all digests.

3.3 Kernel and Memory

`sha256_kernel` is a thin wrapper; heavy work lives in `__device__ inline` functions (`sha256_transform`, `sha256_hash`):

```
i = threadIdx.x + blockIdx.x * blockDim.x;
if (i < num_messages)
    sha256_hash(messages + offsets[i], lengths[i],
                digests + i * 32);
```

Table 2: Device buffers per `sha256_gpu_hash()` call.

Pointer	Size	Purpose
`d_messages`	`total_bytes`	Packed input bytes
`d_offsets`	$N \times 4$ bytes	Start index per message
`d_lengths`	$N \times 4$ bytes	Length per message
`d_digests`	$N \times 32$	Output digests

Memory hierarchy: global memory holds packed payloads, index arrays, and digests; SHA-256 round constants $K[0..63]$ sit in `__constant__` memory for efficient broadcast; per-thread hash state, padding buffer (up to 128 bytes), and round workspace use registers and local memory. Shared memory is unused because threads do not cooperate.

Buffers are freed in a `cleanup()` path on both success and failure. Launch uses the following before the D2H copy:

```
sha256_kernel<<<numBlocks, 256>>>(...);
cudaGetLastError();
cudaDeviceSynchronize();
```

The host API throws `std::runtime_error` on CUDA failure.

3.4 Key Design Decisions

- **Packed buffer + index arrays** — one H2D copy for payload instead of N per-message device pointers.
- **Constant memory for K[]** — round constants read by every thread.
- **Guard `i < num_messages`** — handles arbitrary N without host-side grid padding logic.
- **`size_t` digest offsets** (`i * 32`) — safe past ~67M messages on 32-bit indexing.
- **Correctness before speed** — smoke tests, edge-case validation, and full-dataset compare precede trusted timing.

4 Performance Evaluation and Benchmark Outcomes

4.1 Setup

Table 3: Benchmark environment (from `results/benchmark.md`).

Item	Value
GPU	NVIDIA GeForce RTX 4060 Laptop (24 SMs, 8.6 GB, sm_89)
CUDA	Runtime 13.2, driver 13.2 (nvcc 13.0)
Host compiler	MSVC 14.44 (VS 2022 Build Tools)
OS	Windows 11
CPU baseline	Single-threaded scalar SHA-256 (FIPS 180-4)

4.2 Benchmark Harness and Methodology

Scale benchmarks use `results/scale_benchmark.cu`, which splits GPU time with CUDA events (H2D, kernel, D2H), warms up for ~300 ms, repeats each point 10–50 times, and reports medians, min/max, and standard deviation. Each point checks GPU digests against the CPU reference; during the 1M real-dataset run, `expected_digests.bin` is also checked. To generate the run log, use the `results/benchmark` option; output is written to `results/benchmark_run.txt`. Development validation uses OpenSSL (`validate.cpp`); the scale harness uses a self-contained scalar CPU SHA-256 and checks against NIST vectors on startup.

4.3 Correctness

Table 4: Correctness gates from scale benchmark run (all passed).

Check	Result
Host SHA-256 vs. NIST vectors	Match
GPU vs. CPU reference (1M real + synthetic sweep)	0 mismatches
GPU vs. `expected_digests.bin` (1M real `data/`)	Byte-for-byte

4.4 Real Dataset: 1M Messages

Table 5 breaks down end-to-end GPU time on the Karan-generated `data/` set. PCIe transfer dominates (~70%); kernel compute is ~29% of wall time.

Table 5: GPU time breakdown, 1M real messages (median).

Stage	ms	Share
H2D copy	3.59	31%
Kernel	3.39	29%
D2H copy	4.54	39%
Total	**11.55**	**100%**

Table 6: CPU vs GPU throughput, 1M real messages (median).

Metric	CPU	GPU	Speedup
Time (ms)	212.2	11.55 (e2e) / 3.39 (kernel)	18.4× / 62.5×
Throughput	4.7 M hash/s	86.6 M hash/s (e2e)	18.4×
Kernel bandwidth	—	9.43 GB/s	—

4.5 Scaling Sweep (Synthetic)

4.6 Block-Size Sweep (N=1,000,000, Kernel-Only)

Reading the results:

- Below ~100K messages, launch and PCIe overhead cap end-to-end speedup at ~11×; above 100K the GPU saturates at ~16–18× e2e and ~55–63× kernel-only.
- Transfers account for ~70% of wall time at 1M messages—the PCIe bus is part of the performance model.
- Block-size tuning (64–1024) spans only a ~7% kernel-time band; the workload is memory- and PCIe-bound, not occupancy-limited.
- Variable message lengths cause warp divergence (different compression-round counts per thread), but this is secondary to transfer overhead.

Table 7: Scaling sweep (median, ~32-byte messages). Source: `results/benchmark_run.txt`.

N	**Input**	**CPU ms**	**GPU e2e**	**GPU kernel**	**e2e ×**	**kernel ×**	**OK**
10K	0.32 MB	2.16	0.190	0.041	11.4×	52.8×	✓
100K	3.25 MB	20.13	1.239	0.371	16.3×	54.3×	✓
1M	32.5 MB	202.30	11.78	3.438	17.2×	58.8×	✓
10M	325 MB	2035.5	119.8	34.09	17.0×	59.7×	✓

Table 8: Block-size sensitivity (median kernel time).

Threads/block	**Kernel (ms)**	**Hashes/s**	**GB/s**
64	3.28	305 M	9.91
128	3.39	295 M	9.60
256	3.47	288 M	9.36
512	3.53	283 M	9.20
1024	3.49	287 M	9.31

5 Conclusion

We used a one-thread-per-message mapping and implemented batch SHA-256 on CUDA: inputs and outputs remove synchronizations and races, and allocation is taken care of by a host engine. Transfers, launches and cleanups. The fundamental ideas of GPU programming are: 1. Find the level of parallelism that is proper for the data independence at the message level; not within one hash. 2. Do not measure kernel time, measure entire pipeline (copy + kernel + copy). 3. Don't believe the benchmarks until they've been verified on the GPU path.

The memory space needed for the batch API call is allocated each time it is called. Overlapping transfers with compute (pinned memory, CUDA streams) is the natural next step, rather than micro-optimizing SHA-256 rounds.